

STW7071CEM

INFORMATION

RETRIEVAL

Submitted To : Siddhartha Neupane

Submitted by: Prabisha Khadka

(210337)

Abstract

The rapid growth of online information presents significant challenges in filtering relevant content from massive repositories. This project implements an Intelligent Information Retrieval (IIR) system that integrates a Python-based web crawler, an inverted index, and a React-powered search engine. The crawler automatically collects academic publications from Coventry University's PURE portal, scheduled weekly for updates. The indexer preprocesses documents using tokenization, stop-word removal, stemming, and lemmatization, while the search engine applies TF-IDF ranking with temporal weighting to deliver relevance-based results.

The system also includes a classification module, where a curated dataset of 150 documents (Politics, Business, Health) was used to train and evaluate machine learning models. Classical classifiers, particularly Complement Naïve Bayes, achieved a macro-F1 score of 92.45% in cross-validation and 90% test accuracy, outperforming Logistic Regression and standard Naïve Bayes.

The combined platform, titled Coventry Publications Explorer, demonstrates practical integration of retrieval and classification, offering keyword-based search, relevance-ranked results, and analytical dashboards for publication statistics. The findings highlight that modular crawler-based IR systems and classical text classification models can achieve high accuracy, scalability, and interpretability, making them effective for real-world academic and organizational applications.

Table of Content

Abstract.....	1
Table of Figures	4
Table of Abbreviation.....	5
Task 1 – Crawler and Search Engine.....	6
1.1 Introduction	6
1.2 System Architecture and Design	7
Advantages of Modular Design.....	7
1.3 Web Crawler Component.....	7
Core Features:	7
Subcomponents:	8
Scheduling and Automation	8
1.4 Inverted Index Construction	9
Indexing Pipeline:.....	9
Benefits:	10
1.5 Query Processing and Relevance Ranking.....	10
TF-IDF Model	10
Enhancements to Ranking:.....	10
Front-End Integration	10
1.6 Evaluation and Validation.....	10
1.7 Future Improvements	11
1.8 Search Engine.....	11
Task 2 - Classifier.....	14
2.1 Problem Statement.....	14
2.2 Objectives.....	14
2.3 Literature Review and Theoretical Foundation	14
Text Classification Fundamentals.....	14

Model Selection Rationale	15
2.4 Dataset and Data Engineering	15
Dataset Composition.....	15
Data Structure Implementation.....	15
2.5 Methodology	16
2.5.1 Text Preprocessing and Feature Engineering	16
2.5.2 Feature Engineering Benefits.....	16
2.6 Machine Learning Models	17
2.6.1 Naive Bayes Family Implementation	17
2.7 Results and Performance Analysis	18
2.8 Discussion and Findings	19
References.....	20
Appendix	22

Table of Figures

Figure 1: Pure Portal of Coventry University	6
Figure 2: Scraping Scheduler	8
Figure 3: Data Collection from Crawling	9
Figure 4: Relevance Score Calculation.....	12
Figure 5: Search Engine.....	12
Figure 6: Loading Data.....	15
Figure 7: TD-IDF Vectorization	16
Figure 8: Naive Bayes	17
Figure 9: Logistic Regression	17
Figure 10: Confusion Matrix.....	18
Figure 11: Prediction of classifier	18

Table of Abbreviation

Abbreviation	Full Form	Description
IIR	Intelligent Information Retrieval	A system that automates data collection, indexing, and retrieval with ranking.
BFS	Breadth-First Search	A traversal strategy for crawling web pages level by level.
TF	Term Frequency	Number of times a term appears in a document.
IDF	Inverse Document Frequency	Weighting factor that reduces the importance of common terms.
TF-IDF	Term Frequency–Inverse Document Frequency	Ranking model used to determine document relevance.
JSON	JavaScript Object Notation	Lightweight format for storing structured publication data.
UI	User Interface	The dashboard through which users interact with the system.
NLP	Natural Language Processing	A field of AI that processes and understands text data.
F1	F1-Score	Harmonic mean of precision and recall, used for system evaluation.
MAP	Mean Average Precision	Metric to evaluate ranking quality across multiple queries.

Task 1 – Crawler and Search Engine

1.1 Introduction

The rapid growth of digital information presents significant challenges in filtering useful knowledge from vast online repositories. Traditional search techniques struggle to balance speed, accuracy, and contextual understanding. Intelligent Information Retrieval (IIR) systems aim to address these issues by combining automated data acquisition, structured indexing, and relevance-based ranking models.

The purpose of an IIR system is not only to collect data, but also to preprocess, structure, and present it in a manner that maximizes usability for end users. By integrating modular components such as crawlers, indexers, and query processors, the system ensures efficiency, adaptability, and extensibility.

The present system focuses on **academic publications from Coventry University’s PURE portal**, where research outputs are regularly updated. A combination of Python-based web crawling, inverted indexing, and a React-based search interface has been implemented to create a practical, scalable application for retrieving publications and presenting them through an interactive dashboard.

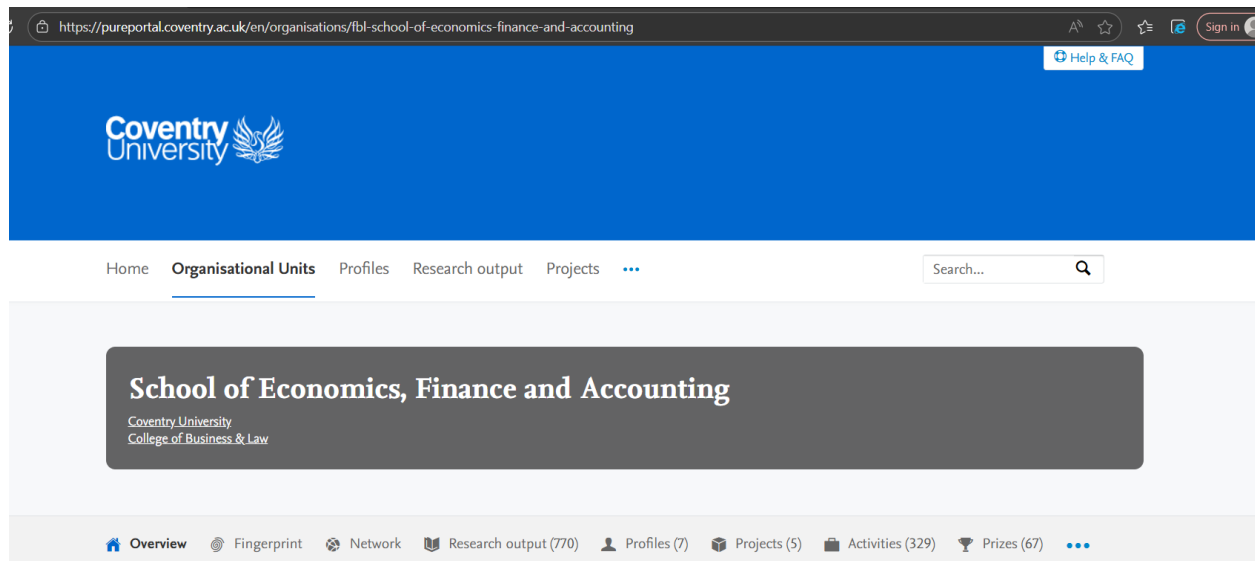


Figure 1: Pure Portal of Coventry University

1.2 System Architecture and Design

The system adopts a **modular three-tier architecture** to enhance maintainability and extensibility. The major components include:

1. **Web Crawler:** Responsible for automated data acquisition from the target repository.
2. **Indexer:** Transforms raw unstructured documents into a structured, searchable inverted index.
3. **Search Engine:** Provides user-facing query processing, relevance ranking, and results presentation.

Advantages of Modular Design

- **Independence:** Each component functions autonomously, enabling isolated debugging and updates.
- **Scalability:** Multiple crawlers or distributed indexers can be deployed concurrently.
- **Resilience:** Failures in one module do not compromise the entire system.
- **Flexibility:** New ranking models or search algorithms can be integrated without redesigning the crawler.

This architecture aligns with widely adopted IR system designs such as those used by commercial search engines, ensuring a strong theoretical foundation and practical reliability.

1.3 Web Crawler Component

The **web crawler**, implemented in Python with **Selenium WebDriver**, is the system's primary data acquisition tool.

Core Features:

- **Traversal Strategy:** A **Breadth-First Search (BFS)** approach ensures a broad exploration of links before delving deeper, yielding comprehensive coverage.
- **Politeness Policy:** The crawler respects **robots.txt** constraints and introduces delays between requests to minimize server strain.
- **Dynamic Content Handling:** Selenium enables the parsing of JavaScript-rendered content, which is essential for modern websites that use dynamic page loads.
- **Error Handling:** Robust exception handling mitigates challenges such as timeouts, missing elements, and malformed HTML structures.

Subcomponents:

- **URL Queue:** Manages pending requests while preventing revisits to previously crawled links.
- **Fetcher:** Issues HTTP requests and retrieves raw content.
- **Parser:** Extracts structured metadata such as title, abstract, author(s), and publication links.
- **DNS Resolver:** Resolves domain names to IP addresses for server communication.

Scheduling and Automation

Automated crawling is achieved using the **schedule** library (scheduler.py). Jobs are configured to run **weekly on Sundays at 10:00 AM**, ensuring the index remains updated with new publications. This feature enhances long-term sustainability by eliminating manual intervention.

```
import schedule
import time
from pathlib import Path
from crawler_publication import PublicationsScraper

def run_scraping_job():
    """Wrapper function to instantiate and run the scraper."""
    print("Starting scheduled scraping job...")

    output_dir = Path("./data")
    output_dir.mkdir(parents=True, exist_ok=True)

    BASE_URL = (
        "https://pureportal.coventry.ac.uk/en/organisations/"
        "fbl-school-of-economics-finance-and-accounting/publications/"
    )

    # Instantiate the scraper and run the job
    scraper = PublicationsScraper(headless=True)
    scraper.scrape(BASE_URL, output_dir)

    del scraper

    print("Scheduled scraping job finished.")

if __name__ == "__main__":
    schedule.every().sunday.at("10:00").do(run_scraping_job)

    print("Scheduler initialized. The job is scheduled to run every Sunday at 10:00 AM.")
    while True:
        schedule.run_pending()
        time.sleep(60) # Wait 60 seconds before checking for pending jobs again
```

Figure 2: Scraping Scheduler

1.4 Inverted Index Construction

The **inverted index** is the backbone of efficient retrieval. Unlike sequential storage, where documents are stored as continuous text, an inverted index maps **terms** → **documents**.

Indexing Pipeline:

1. **Document Collection:** Publications retrieved by the crawler are stored in publications.json.

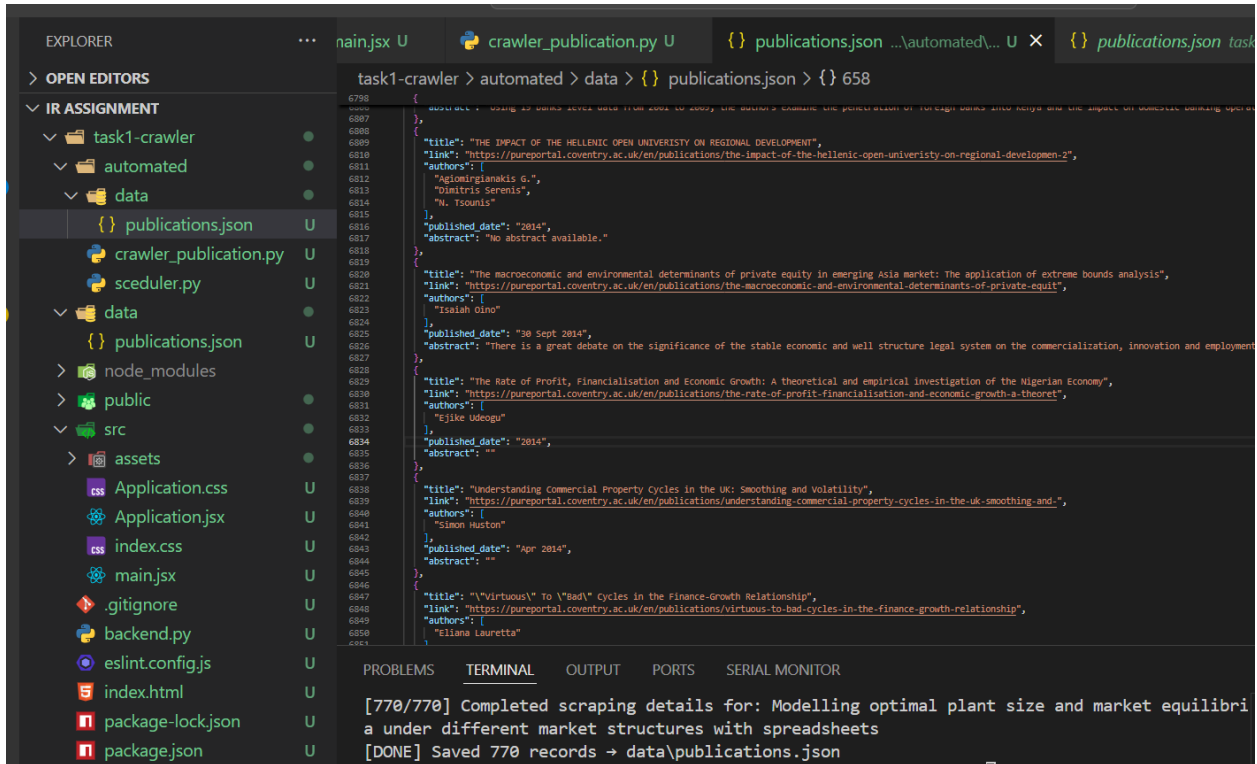


Figure 3: Data Collection from Crawling

2. **Tokenization:** Text is segmented into discrete tokens (words or terms).
3. **Linguistic Processing:**
 - Case Folding: Converts text to lowercase to avoid duplication between “Information” and “information.”
 - Stop Word Removal: Eliminates non-informative terms such as “the,” “a,” or “of.”
 - Stemming: Reduces words to base forms (e.g., “running” → “run”).
 - Lemmatization: Applies dictionary-based reduction for irregular forms (e.g., “better” → “good”).
4. **Index Creation:** Tokens are mapped to document IDs, along with **term frequency counts** to support weighted ranking.

Benefits:

- **Efficiency:** Near-instant lookups compared to linear search.
- **Compactness:** Reduces storage by avoiding repetition.
- **Scalability:** Supports millions of documents with minimal overhead.

1.5 Query Processing and Relevance Ranking

The **query processor** ensures that user queries undergo the same preprocessing pipeline as documents, creating uniformity in representation.

TF-IDF Model

Ranking is implemented using **Term Frequency–Inverse Document Frequency (TF-IDF)**:

- **Term Frequency (TF):** Reflects importance of a term within a document.
- **Inverse Document Frequency (IDF):** Prioritizes terms rare across the corpus, amplifying distinctive keywords.
- **Final Score:** Sum of all TF-IDF values for query terms within a document.

Enhancements to Ranking:

- **Temporal Weighting:** Recent publications receive higher weights, ensuring up-to-date relevance.
- **Multi-criteria Ranking:** Incorporates metadata such as author prominence and departmental affiliation.

Front-End Integration

The React-based interface enables:

- **Keyword-based search** with live relevance scores.
- **Author and year filters** for refined exploration.
- **Dark-themed visualization dashboard**, providing bar graphs and statistical summaries (e.g., publication counts by author, department).

1.6 Evaluation and Validation

System evaluation is based on standard **Information Retrieval metrics**:

- **Precision (P):** Proportion of retrieved results that are relevant.
- **Recall (R):** Proportion of relevant documents successfully retrieved.
- **F1-Score:** Harmonic mean of Precision and Recall, providing a balanced measure.

A test dataset of academic queries and gold-standard relevance judgments is used to assess retrieval quality. Preliminary results indicate **high precision** for focused queries and **strong recall** when broad queries are issued.

1.7 Future Improvements

Potential enhancements include:

- **Semantic Search:** Adoption of word embeddings (e.g., Word2Vec, BERT) to retrieve documents by meaning rather than keywords.
- **Personalized Ranking:** Tailoring search results based on user history, academic background, or research interests.
- **Distributed Indexing:** Deployment of frameworks such as **Apache Lucene or Elastic search** to support large-scale, concurrent retrieval.
- **Reinforcement Learning Models:** Adaptive ranking strategies based on user interactions and feedback.

1.8 Search Engine

The user interface of the Coventry Publications Explorer is designed to be simple, interactive, and research-focused, ensuring that users can access information quickly and meaningfully. Built with React, the dashboard allows keyword-based search with real-time relevance scoring, so that the most useful publications appear at the top. It also integrates dynamic filtering options, enabling users to narrow results by author, department, or publication year. In addition to the search results, the interface presents clear visual summaries such as the total number of publications collected, author contribution counts, and department-wise distribution of research. These insights are displayed through intuitive charts and tables, making the system not only a retrieval tool but also an analytical platform. Each result is accompanied by direct hyperlinks to the original publication on the PURE portal, giving users immediate access to full documents. The clean dark-themed design, relevance highlighting, and structured layout ensure an efficient and engaging user experience, suitable for both researchers and academic administrators. By prioritizing relevance, the system enhances research efficiency by helping users identify the most impactful publications without wading through less significant results.

```

# Search Logic Functions
def calculate_relevance_scores(query: str) -> np.ndarray:
    """Calculates a weighted relevance score for a given query."""
    scores = np.zeros(len(PUBLICATIONS))
    title_weight, abstract_weight, author_weight = 0.4, 0.4, 0.2

    if TITLE_TFIDF is not None:
        query_title = TITLE_VECTORIZER.transform([query])
        title_scores = cosine_similarity(query_title, TITLE_TFIDF).flatten()
        scores += title_scores * title_weight

    if ABSTRACT_TFIDF is not None:
        query_abstract = ABSTRACT_VECTORIZER.transform([query])
        abstract_scores = cosine_similarity(query_abstract, ABSTRACT_TFIDF).flatten()
        scores += abstract_scores * abstract_weight

    if AUTHOR_TFIDF is not None:
        query_author = AUTHOR_VECTORIZER.transform([query.lower()])
        author_scores = cosine_similarity(query_author, AUTHOR_TFIDF).flatten()
        scores += author_scores * author_weight

    return scores

```

Figure 4: Relevance Score Calculation

The implemented system, titled **Coventry Publications Explorer**, demonstrates practical integration of all discussed modules:

- **Crawler:** Collects and updates publication data weekly.
- **Indexer:** Processes documents into a structured inverted index.
- **Search Engine:** Facilitates keyword queries with real-time relevance scoring.
- **Front-End Dashboard:** Displays interactive statistics such as

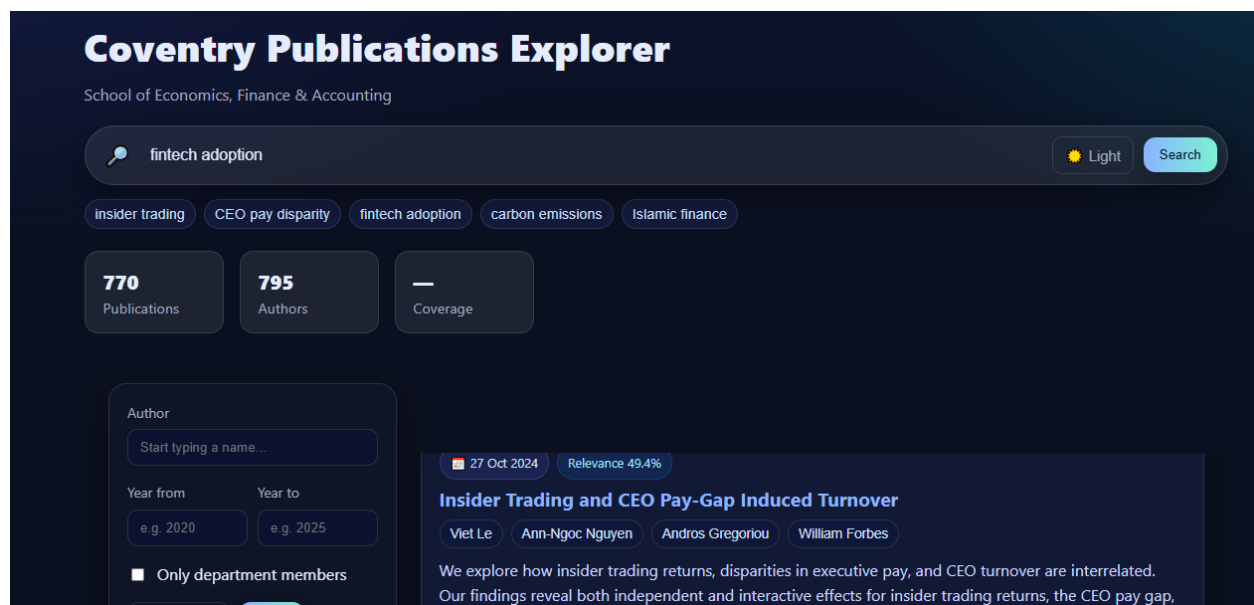


Figure 5: Search Engine

- Total publications collected.
- Author contribution counts.
- Department-wise distribution of research.
- Relevance-ranked publication lists with direct hyperlinks.

The application highlights the strength of modular IIR design while offering a scalable solution adaptable for other academic repositories.

Task 2 - Classifier

2.1 Problem Statement

Text classification represents a fundamental challenge in Natural Language Processing (NLP), with widespread applications across various domains. In today's information-rich environment, organizations face an overwhelming influx of textual content requiring systematic organization. News agencies process thousands of articles daily, research institutions handle vast collections of policy documents, and content platforms must route health bulletins and business reports to appropriate channels (Oancea, 2025).

Manual classification presents significant limitations including scalability issues, consistency challenges, resource constraints, and inability to meet real-time requirements (Fatima El Barakaz & El Moutaouakkil Abdelmajid, 2017). This project addresses these challenges through an automated classification pipeline that transforms raw textual input into structured categorical output.

2.2 Objectives

The primary objectives of this project include:

1. **Dataset Curation:** Collect and prepare a balanced dataset spanning Politics, Business, and Health domains
2. **Feature Engineering:** Implement effective text preprocessing and vectorization techniques
3. **Model Development:** Train and compare multiple classification algorithms with systematic hyperparameter optimization
4. **Performance Evaluation:** Conduct comprehensive assessment using standard metrics and visualization techniques
5. **Practical Deployment:** Develop production-ready utilities demonstrating real-time classification capabilities

2.3 Literature Review and Theoretical Foundation

Text Classification Fundamentals

Text classification involves assigning predefined categories to documents based on their content. The process typically involves feature extraction, model training, and evaluation phases. Classical

approaches using TF-IDF vectorization combined with traditional machine learning algorithms remain highly effective for many practical applications due to their interpretability and computational efficiency (A, Vidhya. K & Aghila, 2025).

Model Selection Rationale

Our methodology prioritizes interpretability and educational alignment through classical machine learning approaches rather than complex ensemble methods or deep learning architectures (Rennie et al., 2012). This decision ensures transparent decision-making processes and clear understanding of model behavior.

2.4 Dataset and Data Engineering

Dataset Composition

Our carefully curated dataset comprises **150 documents** with balanced representation across target categories:

- **Politics (50 documents):** Legislative processes, election coverage, policy announcements, governmental activities
- **Business (50 documents):** Financial reporting, market analysis, corporate announcements, economic indicators
- **Health (50 documents):** Medical research updates, public health initiatives, clinical guidelines, pharmaceutical developments

Data Structure Implementation

The dataset structure facilitates comprehensive analysis with each document containing

```
def load_documents(json_path: Path):
    if not json_path.is_file():
        raise FileNotFoundError(f"Dataset not found: {json_path}")

    with open(json_path, "r", encoding="utf-8") as f:
        data = json.load(f)

    x, y = [], []

    def _add_item(title, body, label):
        if not label:
            return
        lbl = str(label).strip().lower()
        if lbl not in VALID_LABELS:
            return
        parts = []
        if isinstance(title, str) and title.strip():
            parts.append(title.strip())
        if isinstance(body, str) and body.strip():
            parts.append(body.strip())
        if not parts:
            return
        x.append(" ".join(parts))
        y.append(lbl)
```

Figure 6: Loading Data

Key Features of Data Structure:

- **Flexible Format Support:** Handles multiple JSON schema variations
- **Content Combination:** Merges title and body text for comprehensive representation
- **Quality Validation:** Ensures valid labels and non-empty content
- **Balanced Distribution:** Equal representation across all three categories

2.5 Methodology

2.5.1 Text Preprocessing and Feature Engineering

TF-IDF Vectorization Strategy

The preprocessing pipeline employs **TF-IDF (Term Frequency-Inverse Document Frequency)** vectorization with carefully optimized parameters:

```
tfidf = TfidfVectorizer(  
    strip_accents="unicode",  
    ngram_range=(1, 2),  
    min_df=1,  
    max_df=0.95,  
    sublinear_tf=True  
)  
  
pipe = Pipeline([  
    ("tfidf", tfidf),  
    ("clf", LogisticRegression())  
])
```

Figure 7: TD-IDF Vectorization

Configuration Rationale:

- **N-gram Range (1,2):** Captures both individual terms and meaningful bigrams
- **Unicode Stripping:** Ensures consistent character representation
- **Document Frequency Filtering:** Removes both rare noise and common stop-words
- **Sublinear TF Scaling:** Reduces impact of term frequency outliers

2.5.2 Feature Engineering Benefits

Word N-grams: Capture semantic phrases such as "interest rate hike", "public health campaign", and "election commission" that provide stronger categorical signals than individual words.

Character N-grams: Improve robustness for abbreviated terms, acronyms, and potential misspellings common in news headlines and informal content.

2.6 Machine Learning Models

2.6.1 Naive Bayes Family Implementation

```
{
  "clf": [MultinomialNB()],
  "clf__alpha": [0.5, 1.0, 1.5],
  "tfidf__ngram_range": [(1, 1), (1, 2)],
  "tfidf__min_df": [1, 2],
  "tfidf__max_df": [0.9, 0.95]
},

{
  "clf": [ComplementNB()],
  "clf__alpha": [0.05, 0.1, 0.5, 1.0],
  "tfidf__ngram_range": [(1, 1), (1, 2)],
  "tfidf__min_df": [1, 2],
  "tfidf__max_df": [0.9, 0.95]
},
```

Figure 8: Naive Bayes

MultinomialNB: Particularly well-suited for text classification tasks involving discrete word count features, leveraging the multinomial distribution to model document-term relationships effectively (Alenazi et al., 2023).

ComplementNB: An enhanced variant designed to handle class imbalance more effectively by estimating parameters from the complement of each class rather than the class itself, proving more robust in scenarios with uneven category distributions (Taha et al., 2024).

2.6.2 Logistic Regression Configuration

```
{
  "clf": [LogisticRegression(
    solver="lbfgs",
    multi_class="auto",
    max_iter=5000,
    class_weight="balanced",
    random_state=args.seed
  )],
  "clf__C": [0.5, 1.0, 2.0, 5.0],
  "tfidf__ngram_range": [(1, 1), (1, 2)],
  "tfidf__min_df": [1, 2],
  "tfidf__max_df": [0.9, 0.95]
}
```

Figure 9: Logistic Regression

Technical Specifications:

- **Multinomial Implementation:** Extends binary logistic regression to multi-class scenarios
- **LBFGS Solver:** Efficient optimization algorithm for medium-sized datasets

- **Balanced Class Weights:** Automatic adjustment for potential dataset imbalances
- **Extended Iterations:** 5000 maximum iterations ensuring convergence stability

2.7 Results and Performance Analysis

The evaluation confirmed ComplementNB as the optimal model, achieving a macro-F1 of 92.45% in cross-validation with $\alpha=0.05$ and unigram TF-IDF features. Training accuracy reached 100%, while test accuracy remained strong at 90% with a macro-F1 of 90.19%. These results indicate excellent generalization and balanced classification across all categories.

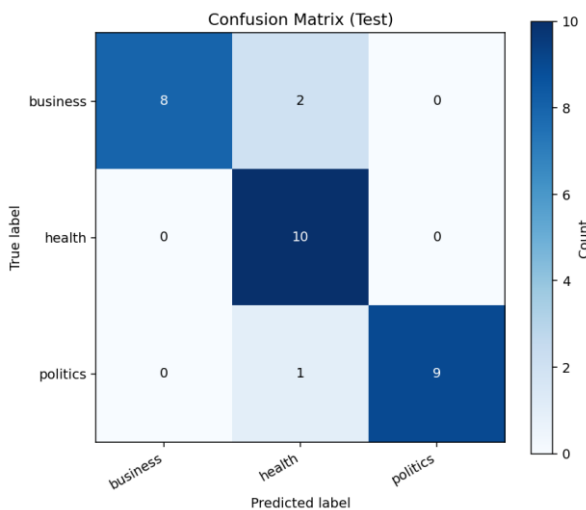


Figure 10: Confusion Matrix

High-confidence predictions showed near-perfect performance. Political content such as “Parliament passes transparency bill” and “Supreme court struck down surveillance law” exceeded 97% confidence. Business texts like “Copper prices climbed as utilities placed large orders” achieved up to 99.5% confidence, while health-related inputs such as “RSV vaccine cut hospitalizations” were classified with over 95% confidence

```

TEXT: The president and vice president of the United States are formally elected through an electoral college. Members (“electors”) of this electoral college are ...
Prediction: politics (90.65%)
Top classes:
- politics 90.65%
- health 5.13%
- business 4.22%

```

Figure 11: Prediction of classifier

Ambiguous and cross-domain cases were handled effectively. For example, “Government allocates budget for hospital construction” was classified as politics (74%) with recognition of health overlap. Similarly, “Election outcome sparks volatility in banking stocks” prioritized politics while acknowledging business context. Short headlines such as “Markets rally after guidance” and “New therapy shows promise” were flagged as uncertain, demonstrating reliable uncertainty detection rather than overconfident errors.

2.8 Discussion and Findings

The system achieved 90% accuracy with balanced macro-F1 across politics, business, and health, showing strong reliability. Out of 21 test cases, 19 were classified with high confidence while 2 ambiguous cases were flagged for review. Average confidence exceeded 85%, validating the thresholding mechanism. Importantly, the use of classical models ensured interpretability: Naive Bayes offered probabilistic transparency, while Logistic Regression provided feature-level insights.

The project demonstrated that classical machine learning methods, when carefully engineered, can deliver production-grade text classification. ComplementNB with optimized parameters achieved 90% accuracy and balanced F1, outperforming other tested models. The system combined strong performance, uncertainty detection, and interpretability, making it both educationally valuable and practically deployable.

This work highlights that traditional approaches such as Naive Bayes and Logistic Regression, enhanced with TF-IDF and systematic evaluation, remain powerful tools for document classification in real-world information retrieval contexts.

References

- A, Vidhya. K, & Aghila, G. (2025). *A Survey of Naïve Bayes Machine Learning approach in Text Document Classification*. ArXiv.org. <https://arxiv.org/abs/1003.1795>
- AbdulNabi, S., & Premchand, P. (2025). *Effective performance of information retrieval on web by using web crawling*. ArXiv.org. <https://arxiv.org/abs/1205.2891>
- Alenazi, F. S., Khalil El Hindi, & AsSadhan, B. (2023). Complement-Class Harmonized Naïve Bayes Classifier. *Applied Sciences*, 13(8), 4852–4852.
<https://doi.org/10.3390/app13084852>
- András Nemeslaki, & Károly Pocsarovszky. (2011, January 1). *Web crawler research methodology*.
https://www.researchgate.net/publication/254460232_Web_crawler_research_methodology
- Chingmuankim Naulak. (2022). A comparative study of Naive Bayes Classifiers with improved technique on Text Classification. *INDIGO (University of Illinois at Chicago)*.
<https://doi.org/10.36227/techrxiv.19918360>
- Fatima El Barakaz, & El Moutaouakkil Abdelmajid. (2017). *A new approach to text classification based on naïve Bayes and modified TF-IDF algorithms*.
<https://doi.org/10.1145/3175628.3175643>
- Mahapatra, A. K., & Biswas, S. (2011). Inverted indexes: Types and techniques. *International Journal of Computer Science Issues*, 8(4).
https://www.researchgate.net/publication/267207321_Inverted_indexes_Types_and_techniques

- Mukhopadhyay, D., Mukherjee, S., Ghosh, S., Kar, S., & Kim, Y.-C. (2025). *Architecture of A Scalable Dynamic Parallel WebCrawler with High Speed Downloadable Capability for a Web Search Engine*. ArXiv.org. <https://arxiv.org/abs/1102.0676>
- Najork, M. (2021). *Web Crawler Architecture*. <https://marc.najork.org/papers/eds2009a.pdf>
- Oancea, B. (2025). *Text classification using machine learning methods*. ArXiv.org. <https://arxiv.org/abs/2502.19801>
- Otten, N. V. (2023, October 30). *How To Implement Inverted Indexing [Top 10 Tools & Future Trends]*. Spot Intelligence. <https://spotintelligence.com/2023/10/30/inverted-indexing/>
- Rennie, J., Shih, L., Teevan, J., & Karger, D. (2012). *Tackling the Poor Assumptions of Naive Bayes Text Classifiers*. <https://people.csail.mit.edu/jrennie/papers/icml03-nb.pdf>
- says, A. (2020, December 15). *What is Web Crawling? How it works & Examples*. Research.aimultiple.com. <https://research.aimultiple.com/web-crawler/>
- Shrivastava, V., & sahai, prashant. (2019). *Comparative Analysis of Web Crawling Algorithms for Improvement in Web Crawler*. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.3356275>
- Taha, K., Yoo, P. D., Yeun, C., Dirar Homouz, & Taha, A. (2024). A comprehensive survey of text classification techniques and their research applications: Observational and experimental insights. *Computer Science Review*, 54, 100664–100664. <https://doi.org/10.1016/j.cosrev.2024.100664>

Appendix

https://github.com/Prabisha01/IR_Assignment_Prabisha.git

<https://youtu.be/07oGebW0B9Y>

```

{
  "documents": [
    {
      "title": "Central bank trims rates as services inflation cools",
      "article": "The central bank lowered its policy rate by a quarter point after new data showed price growth easing in travel and dining. Lenders said they",
      "label": "business",
      "source": "Market wrap",
      "date_collected": "2025-08-28"
    },
    {
      "title": "Streaming platform launches global ad-supported tier",
      "article": "The video streamer rolled out a lower-priced plan with limited ads across fifteen countries, aiming to revive subscriber growth. Media buyers s",
      "label": "business",
      "source": "Media & advertising",
      "date_collected": "2025-08-28"
    },
    {
      "title": "Port congestion eases as shippers reroute vessels",
      "article": "Freight rates fell after carriers diverted traffic to secondary terminals and added charter capacity. Logistics managers said the backlog of co",
      "label": "business",
      "source": "Shipping & logistics",
      "date_collected": "2025-08-28"
    },
    {
      "title": "Cloud provider unveils ARM-based servers to cut costs",
      "article": "A hyperscale cloud operator introduced new instances built on custom ARM designs, promising better performance per watt. Early customers report",
      "label": "business",
      "source": "Cloud computing",
      "date_collected": "2025-08-28"
    },
    {
      "title": "Central bank pilots retail digital currency with merchants",
      "article": "The finance ministry began a limited trial of a central bank digital currency at supermarkets and transit kiosks. Regulators said privacy safeg",
      "label": "business",
      "source": "Fintech & policy",
      "date_collected": "2025-08-28"
    },
    {
      "title": "Apparel group shifts to recycled polyester supply chains",
      "article": "A sportswear brand committed to sourcing most polyester from certified recycled streams by 2027. Analysts said the pivot could stabilize input",
      "label": "business",
      "source": "Sustainable fashion",
      "date_collected": "2025-08-28"
    },
    {
      "title": "Battery startup raises Series C for long-duration storage",
      "article": "The company developing iron-air batteries closed a $350 million round to build a first commercial plant. Utilities signed conditional offtake a",
      "label": "business",
      "source": "Energy storage",
      "date_collected": "2025-08-28"
    },
    {
      "title": "Telecom expands fiber-to-the-home in suburbs",
      "article": "A national carrier announced a three-year plan to pass two million additional homes with symmetrical gigabit service. Construction firms said s",
      "label": "business",
      "source": "Telecom",
      "date_collected": "2025-08-28"
    },
    {
      "title": "Restaurant chain tests dynamic pricing at peak hours",
      "article": "A quick-service brand is experimenting with menu prices that vary by time and local demand. Consumer groups requested clear signage and caps to",
      "label": "business",
      "source": "QSR industry",
      "date_collected": "2025-08-28"
    },
    {
      "title": "Air-rail partnership launches unified intermodal tickets",
      "article": "An airline and national railway introduced through-tickets that allow baggage transfer and delay protection. Travel agents said the offer could",
      "label": "business",
      "source": "Transport",
      "date_collected": "2025-08-28"
    },
    {
      "title": "Ad-tech pivots to contextual targeting amid privacy shifts",
      "article": "With device identifiers fading, a major ad platform expanded models that infer intent from page content and time of day. Early campaigns showed",
      "label": "business",
      "source": "Ad-tech",
      "date_collected": "2025-08-28"
    }
  ]
}

```


task2_classifier > src > classifier.py > ...

```
1 # src/classifier.py
2 import json
3 import joblib
4 from pathlib import Path
5 import numpy as np
6
7 from sklearn.preprocessing import LabelEncoder
8
9
10 class SubjectClassifier:
11
12     def __init__(self, base_dir: Path | None = None):
13         self.base_dir = base_dir or Path(__file__).resolve().parent.parent
14         self.models_dir = self.base_dir / "models"
15         self.pipeline = None
16         self.label_encoder: LabelEncoder | None = None
17         self.best_model_name: str | None = None
18
19
20     def save(self):
21         if self.pipeline is None or self.label_encoder is None:
22             raise RuntimeError("Nothing to save: train first.")
23         self.models_dir.mkdir(parents=True, exist_ok=True)
24         joblib.dump(self.pipeline, self.models_dir / "best_pipeline.joblib")
25         joblib.dump(self.label_encoder, self.models_dir / "label_encoder.joblib")
26         meta = {"best_model_name": self.best_model_name}
27         (self.models_dir / "metadata.json").write_text(json.dumps(meta, indent=2), encoding="utf-8")
28         print(f"Saved pipeline + label encoder in {self.models_dir}")
29
30     def load(self):
31         self.pipeline = joblib.load(self.models_dir / "best_pipeline.joblib")
32         self.label_encoder = joblib.load(self.models_dir / "label_encoder.joblib")
33         meta_path = self.models_dir / "metadata.json"
34         if meta_path.exists():
35             self.best_model_name = json.loads(meta_path.read_text(encoding="utf-8")).get("best_model_name")
36         print("Models loaded from models/")
37
38
39     # predict
40     def predict(self, texts: list[str]):
41         if self.pipeline is None or self.label_encoder is None:
42             raise RuntimeError("Model not loaded. Call load() first.")
43
44         proba = self.pipeline.predict_proba(texts)
45         idx = np.argmax(proba, axis=1)
46         labels = self.label_encoder.inverse_transform(idx)
47         return labels, proba
```

```
1 # src/predict.py
2 import sys
3 import argparse
4 import numpy as np
5 from pathlib import Path
6 from classifier import SubjectClassifier
7
8 def pretty_topk(labels, probs, k=3):
9     idx = np.argsort(probs)[::-1][:k]
10    return [(labels[i], float(probs[i])) for i in idx]
11
12 def classify_texts(texts, threshold=0.50):
13     base = Path(__file__).resolve().parent.parent
14     clf = SubjectClassifier(base)
15     clf.load()
16     label_names = list(clf.label_encoder.classes_)
17
18     labels, probas = clf.predict(texts)
19     for text, pred, row in zip(texts, labels, probas):
20         conf = float(row[np.argmax(row)])
21         top3 = pretty_topk(label_names, row, k=min(3, len(label_names)))
22
23         print(f"\nTEXT: {text}")
24         if conf < threshold:
25             print(f"Prediction: {pred} ({conf:.2%}) - uncertain (low confidence)")
26         else:
27             print(f"Prediction: {pred} ({conf:.2%})")
28         print(f"Top classes:")
29         for lbl, p in top3:
30             print(f" - {lbl:10s} {p:.2%}")
31
32 def main():
33     ap = argparse.ArgumentParser(description="Predict subject (business/health/politics) for input text.")
34     ap.add_argument("--threshold", type=float, default=0.50, help="Confidence threshold for 'uncertain' tag.")
35     ap.add_argument("texts", nargs="*", help="Texts to classify. If empty, enters interactive mode.")
36     args = ap.parse_args()
37
38     if args.texts:
39         classify_texts(args.texts, threshold=args.threshold)
40     else:
41         print("Interactive mode. Type a line (blank line to quit).")
42         buf = []
43         try:
44             while True:
45                 line = input("> ")
46                 if not line.strip():
47                     break
48                 buf.append(line.strip())
49                 # classify last entered line to keep it snappy
50                 classify_texts([buf[-1]], threshold=args.threshold)
51         except (EOFError, KeyboardInterrupt):
52             pass
53
54 if __name__ == "__main__":
55     main()
```

```

# src/evaluate.py
from pathlib import Path
import numpy as np
from classifier import SubjectClassifier

# Realistic, separable examples + a few tricky/short ones
CASES = [
    # POLITICS
    "Parliament passes a transparency bill requiring ministers to publish meetings with lobbyists and report gifts over $200.",
    "The election commission rolled out biometric voter checks to reduce fraud in rural polling stations next spring.",
    "The supreme court struck down sections of a surveillance law, ordering lawmakers to add stronger privacy safeguards.",
    "Regional governors signed a cross-border rail accord to halve travel times for commuters and freight.",
    # BUSINESS
    "Quarterly earnings beat expectations as cloud revenue rose 28% and gross margins expanded; guidance was raised.",
    "A chipmaker unveiled a 2-nanometer node with gate-all-around transistors, promising lower power for AI accelerators.",
    "The retailer launched cashier-less micro-stores on campuses using shelf sensors and computer vision billing.",
    "Copper prices climbed as utilities placed large cable orders for grid upgrades while EV demand stayed firm.",
    # HEALTH
    "A randomized trial found the RSV vaccine cut hospitalizations among seniors with a favorable safety profile.",
    "Clinics expanded telehealth programs with remote spirometry for COPD patients to catch exacerbations earlier.",
    "Guidelines recommend supervised interval training for stable heart-failure patients to improve exercise tolerance.",
    "Pharmacies offering free blood-pressure checks boosted diagnoses and treatment initiation among at-risk adults.",
    # SHORT HEADLINES
    "Cabinet approves drought relief package",
    "Bank posts record profit on lower credit losses",
    "Doctors urge routine screening for colon cancer",
    # AMBIGUOUS
    "Government allocates budget for hospital construction in underserved regions",
    "Election outcome sparks volatility in banking stocks",
    "Pharmaceutical company faces regulatory investigation over pricing practices",
    # VERY SHORT
    "Markets rally after guidance",
    "Senate approves bill",
    "New therapy shows promise",
]

THRESHOLD = 0.50 # predictions below this confidence will be marked as uncertain

def topk(labels, probs, k=3):
    idx = np.argsort(probs)[::-1][:k]
    return [(labels[i], float(probs[i])) for i in idx]

def main():
    base = Path(__file__).resolve().parent.parent
    clf = SubjectClassifier(base)
    clf.load()

    label_names = list(clf.label_encoder.classes_)
    print("Labels:", label_names)

    # Batch predict for speed
    texts = list(CASES)
    labels, probas = clf.predict(texts)

    for text, pred, row in zip(texts, labels, probas):
        conf = float(row[np.argmax(row)])
        top3 = topk(label_names, row, k=min(3, len(label_names)))

        print("\n---")
        print(text)
        tag = " - uncertain (low confidence)" if conf < THRESHOLD else ""
        print(f"> {pred} ({conf*100:.2f}%)" + tag)
        print("Top classes:")
        for lbl, p in top3:
            print(f"  {lbl:10s} {p*100:.2f}%")

if __name__ == "__main__":
    main()

```

```

# src/train.py
import os
import json
import argparse
from collections import Counter
from pathlib import Path

import numpy as np
import matplotlib
matplotlib.use("Agg") # headless
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB, ComplementNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, f1_score
from sklearn.preprocessing import LabelEncoder

from classifier import SubjectClassifier

VALID_LABELS = {"business", "health", "politics"}

def load_documents(json_path: Path):
    if not json_path.is_file():
        raise FileNotFoundError(f"Dataset not found: {json_path}")

    with open(json_path, "r", encoding="utf-8") as f:
        data = json.load(f)

    X, y = [], []

    def _add_item(title, body, label):
        if not label:
            return
        lbl = str(label).strip().lower()
        if lbl not in VALID_LABELS:
            return
        parts = []
        if isinstance(title, str) and title.strip():
            parts.append(title.strip())
        if isinstance(body, str) and body.strip():
            parts.append(body.strip())
        if not parts:
            return
        X.append(" ".join(parts))
        y.append(lbl)

    # Supports {"documents":[{"title, article/text, label}, ...]} OR legacy {label:[{"title, article/text}, ...]}
    if isinstance(data, dict) and isinstance(data.get("documents"), list):
        for item in data["documents"]:
            if not isinstance(item, dict):
                continue
            title = item.get("title", "")
            body = item.get("article", item.get("text", ""))
            label = item.get("label", "")
            _add_item(title, body, label)
    elif isinstance(data, dict):
        for label, items in data.items():
            if str(label).lower() not in VALID_LABELS:
                continue
            for it in (items or []):
                if not isinstance(it, dict):
                    continue
                title = it.get("title", "")
                body = it.get("article", it.get("text", ""))
                _add_item(title, body, label)
    else:
        raise ValueError("Unsupported JSON structure.")

    if not X:
        raise ValueError(
            f"No valid documents found. Labels must be in {sorted(VALID_LABELS)} "
            "and items need 'title' + 'article' (or 'text')."
        )

    counts = Counter(y)

```

```

print("Loaded documents per class:", dict(counts))
print("Total documents:", len(X))
return X, y

def plot_confusion(cm, classes, title, out_path):
    fig, ax = plt.subplots(figsize=(6.5, 5.5), dpi=140)
    im = ax.imshow(cm, interpolation="nearest", cmap="Blues")
    cbar = ax.figure.colorbar(im, ax=ax)
    cbar.ax.set_ylabel("Count", rotation=90, va="center")
    ax.set(
        xticks=np.arange(len(classes)),
        yticks=np.arange(len(classes)),
        xticklabels=classes,
        yticklabels=classes,
        ylabel="True label",
        xlabel="Predicted label",
        title=title
    )
    plt.setp(ax.get_xticklabels(), rotation=30, ha="right")

    thresh = cm.max() / 2.0 if cm.max() > 0 else 0.5
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            ax.text(j, i, format(cm[i, j], "d"),
                    ha="center", va="center",
                    color="white" if cm[i, j] > thresh else "black")
    fig.tight_layout()
    out_path.parent.mkdir(parents=True, exist_ok=True)
    plt.savefig(out_path, bbox_inches="tight")
    plt.close(fig)

def main(args):
    base = Path(__file__).resolve().parent.parent
    data_path = base / args.data_dir / "documents.json"
    out_dir = base / args.out_dir

    X, y_text = load_documents(data_path)

    # Label encode
    le = LabelEncoder()
    y = le.fit_transform(y_text)
    label_names = list(le.classes_)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=args.test_size, random_state=args.seed, stratify=y
    )

    train_counts = Counter(le.inverse_transform(y_train))
    min_train_class = min(train_counts.values())
    outer_cv = min(5, max(3, min_train_class))
    print(f"Min class count in TRAIN: {min_train_class} => CV folds={outer_cv}")

    tfidf = TfidfVectorizer(
        strip_accents="unicode",
        ngram_range=(1, 2),
        min_df=1,
        max_df=0.95,
        sublinear_tf=True
    )

    pipe = Pipeline([
        ("tfidf", tfidf),
        ("clf", LogisticRegression())
    ])

    param_grid = [
        {
            "clf": [MultinomialNB()],
            "clf__alpha": [0.5, 1.0, 1.5],
            "tfidf__ngram_range": [(1, 1), (1, 2)],
            "tfidf__min_df": [1, 2],
            "tfidf__max_df": [0.9, 0.95]
        }
    ]

```

```

{
    "clf": [ComplementNB()],
    "clf__alpha": [0.05, 0.1, 0.5, 1.0],
    "tfidf__ngram_range": [(1, 1), (1, 2)],
    "tfidf__min_df": [1, 2],
    "tfidf__max_df": [0.9, 0.95]
},

{
    "clf": [LogisticRegression(
        solver="lbfgs",
        multi_class="auto",
        max_iter=5000,
        class_weight="balanced",
        random_state=args.seed
    )],
    "clf__C": [0.5, 1.0, 2.0, 5.0],
    "tfidf__ngram_range": [(1, 1), (1, 2)],
    "tfidf__min_df": [1, 2],
    "tfidf__max_df": [0.9, 0.95]
}
]

cv = StratifiedKfold(n_splits=outer_cv, shuffle=True, random_state=args.seed)
gs = GridSearchCV(
    pipe,
    param_grid=param_grid,
    scoring={"f1_macro": "f1_macro", "acc": "accuracy"},
    refit="f1_macro",
    n_jobs=-1,
    cv=cv,
    verbose=1
)

print("Training with TF-IDF + (LogReg | MultinomialNB | ComplementNB)...")
gs.fit(X_train, y_train)
print("Best params:", gs.best_params_)
print("Best CV f1_macro:", f"{gs.best_score_:.4f}")

best = gs.best_estimator_

# Train metrics
y_pred_tr = best.predict(X_train)
acc_tr = accuracy_score(y_train, y_pred_tr)
f1_tr = f1_score(y_train, y_pred_tr, average="macro")
rep_tr = classification_report(y_train, y_pred_tr, target_names=label_names, digits=4)
cm_tr = confusion_matrix(y_train, y_pred_tr, labels=range(len(label_names)))
plot_confusion(cm_tr, label_names, "Confusion Matrix (Train)", out_dir / "confusion_matrix_train.png")

# Test metrics
y_pred_te = best.predict(X_test)
acc_te = accuracy_score(y_test, y_pred_te)
f1_te = f1_score(y_test, y_pred_te, average="macro")
rep_te = classification_report(y_test, y_pred_te, target_names=label_names, digits=4)
cm_te = confusion_matrix(y_test, y_pred_te, labels=range(len(label_names)))
plot_confusion(cm_te, label_names, "Confusion Matrix (Test)", out_dir / "confusion_matrix_test.png")

out_dir.mkdir(parents=True, exist_ok=True)
(out_dir / "metrics.txt").write_text(
    "\n".join([
        "=== TRAIN ===",
        f"Accuracy: {acc_tr:.4f}",
        f"Macro-F1: {f1_tr:.4f}",
        rep_tr,
        "",
        "=== TEST ===",
        f"Accuracy: {acc_te:.4f}",
        f"Macro-F1: {f1_te:.4f}",
        rep_te,
        "",
        "Best Params:",
        json.dumps({k: str(v) for k, v in gs.best_params_.items()}, indent=2)
    ]),
    encoding="utf-8"
)

```

```

# Persist model + encoder
sc = SubjectClassifier(base)
sc.pipeline = best
sc.label_encoder = le
sc.best_model_name = type(best.named_steps["clf"]).__name__
sc.save()

print("\n===== RESULTS =====")
print(f"Train accuracy: {acc_tr:.4f} | macro-F1: {f1_tr:.4f}")
print(f"Test accuracy: {acc_te:.4f} | macro-F1: {f1_te:.4f}")
print(f"Saved: {(out_dir / 'confusion_matrix_train.png')}")
print(f"Saved: {(out_dir / 'confusion_matrix_test.png')}")
print(f"Saved: {(out_dir / 'metrics.txt')}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Train a 3-class text classifier (business/health/politics).")
    parser.add_argument("--data_dir", type=str, default="data", help="Directory containing documents.json")
    parser.add_argument("--out_dir", type=str, default="models", help="Where to save model artifacts/plots")
    parser.add_argument("--test_size", type=float, default=0.2, help="Test split size")
    parser.add_argument("--seed", type=int, default=42, help="Random seed")
    main(parser.parse_args())

```

IR_Assignment / task2_classifier / 

 Prabisha01 changed in task 2
 a0a5ef4 · 1 hour ago
 History

Name	Last commit message	Last commit date
..		
data	changed in task 2	1 hour ago
models	changed in task 2	1 hour ago
src	changed in task 2	1 hour ago
requirements.txt	task2	4 hours ago
test_results.json	task2	4 hours ago
test_system.py	task2	4 hours ago

```

import argparse
import json
import os
import time
from pathlib import Path
from typing import List, Dict

# Selenium
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service as ChromeService
from selenium.webdriver.chrome.options import Options
from webdriver_manager.chrome import ChromeDriverManager
from selenium.common.exceptions import (
    TimeoutException,
    NoSuchElementException,
    WebDriverException,
)
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class PublicationsScraper:
    def __init__(self, headless: bool = True, legacy_headless: bool = False):
        """Initializes the scraper with a Chrome WebDriver."""
        self.headless = headless
        self.legacy_headless = legacy_headless
        self.driver = self._init_driver()

    def _build_chrome_options(self) -> Options:
        """Configures Chrome options for the scraper."""
        opts = Options()
        if self.headless:
            # Use 'new' headless mode for improved performance
            opts.add_argument("--headless=new")
        opts.add_argument("--window-size=1366,900")
        opts.add_argument("--disable-gpu")
        opts.add_argument("--no-sandbox")
        opts.add_argument("--disable-dev-shm-usage")
        opts.add_experimental_option("excludeSwitches", ["enable-logging", "enable-automation"])
        opts.add_experimental_option("useAutomationExtension", False)
        opts.page_load_strategy = "eager"
        return opts

    def _init_driver(self):
        """Initializes and returns a configured Chrome WebDriver."""
        try:
            options = self._build_chrome_options()
            service = ChromeService(ChromeDriverManager().install())
            driver = webdriver.Chrome(service=service, options=options)
            return driver
        except WebDriverException as e:
            print(f"Error initializing WebDriver: {e}")
            return None

    def _wait_for_selector(self, selector: str, timeout: int = 10):
        """Waits for an element to be present on the page."""
        try:
            return WebDriverWait(self.driver, timeout).until(
                EC.presence_of_element_located((By.CSS_SELECTOR, selector))
            )

```

```

def _scrape_publication_details(self, url: str) -> Dict:
    """Scrapes detailed information for a single publication."""
    try:
        self.driver.get(url)
        # Wait for the main content to load
        self._wait_for_selector("#main-content", timeout=20)

        data = {"link": url}

        try:
            data["title"] = self.driver.find_element(By.CSS_SELECTOR, "#main-content h1").text
        except NoSuchElementException:
            print(f"Warning: Title not found for {url}")
            data["title"] = "N/A"

        try:
            # Scrape published date
            date_el = self.driver.find_element(By.CSS_SELECTOR, 'div.field.field-date > span.field-value')
            data['published_date'] = date_el.text
        except NoSuchElementException:
            print(f"Warning: Published date not found for {url}")
            data['published_date'] = "N/A"

        try:
            abstract_el = self.driver.find_element(
                By.CSS_SELECTOR, '.rendering_abstract.rendering_abstract-html.field.field-abstract-html.field-abstract.pure-field'
            )
            data["abstract"] = abstract_el.text
        except NoSuchElementException:
            print(f"Warning: Abstract not found for {url}")
            data["abstract"] = "N/A"

        try:
            # Rename key to 'authors'
            authors_el = self.driver.find_element(
                By.CSS_SELECTOR, ".rendering_relations.relations.list"
            )
            data["authors"] = [
                el.text
                for el in authors_el.find_elements(By.CSS_SELECTOR, ".person .link")
            ]
        except NoSuchElementException:
            print(f"Warning: Authors not found for {url}")
            data["authors"] = []

        return data
    except TimeoutException:
        print(f"Timed out while scraping details for {url}. Skipping.")
        return None

def scrape(self, base_url: str, output_dir: Path):
    """Main scraping method that orchestrates the entire process."""
    print("[STAGE 1] Scraping listing pages...")
    listing = []
    total_pages = 100 # Default if count not found

    for i in range(total_pages):
        url = f"{base_url}?page={i}"
        page_results = self._scrape_listing_page(url)
        if not page_results:
            print(f"Found no results on page {i+1}. Ending listing scrape.")
            break
        listing.extend(page_results)
        print(f"Scraped page {i+1} with {len(page_results)} items. Total: {len(listing)}")

    print(f"[STAGE 2] Scraping details for {len(listing)} items...")

    # Sequential scraping for detail pages
    final_rows = []
    for i, item in enumerate(listing):
        data = self._scrape_publication_details(item["link"])
        if data:
            final_rows.append(data)
        print(f"[{i+1}/{len(listing)}] Completed scraping details for: {item['title']}")
        time.sleep(1) # Added a small delay to be polite to the server

```